

Air University



Operating Systems (Lab)

Year 2026

Ms. Qurrat Ul Ain

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: April 17, 2026

Lab # 08

Write a C Program to demonstrate inter-process communication using shared memory by Following the Algorithm given below:

1. Algorithm for writing to Shared Memory

Directory and File Creation: The terminal shows navigation to the Lab8 directory and the opening of the vi editor to create the writer.c source file.

```
usama@ubuntu:~/Desktop/Lab6$ mkdir Lab8
usama@ubuntu:~/Desktop/Lab6$ cd Lab8
usama@ubuntu:~/Desktop/Lab6/Lab8$ vim writer.c
```

Code: This code segment demonstrates the creation of a shared memory segment using **shmget** and attaching it to the process address space with **shmat**. The program captures user input via **fgets**, writes it into the shared memory segment, and displays the confirmed data before detaching.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#define SHM_SIZE 1024 // Size of shared memory segment
#define SHM_KEY 1234 // Unique key (can also use ftok)

int main() {
    // 2. Create necessary variables
    int shmid;
    char *shmaddr;
    char input[SHM_SIZE];

    // 3. Create a shared memory segment with unique id and read/write permissions
    shmid = shmget(SHM_KEY, SHM_SIZE, IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget");
        exit(EXIT_FAILURE);
    }
    printf("Shared memory created with ID: %d\n", shmid);

    // 4. Attach the process to the shared memory segment
    shmaddr = (char *)shmat(shmid, NULL, 0);
    if (shmaddr == (char *)-1) {
        perror("shmat");
        exit(EXIT_FAILURE);
    }
}
```

```
// 5. Display the address of the memory segment attached with the process
printf("Attached at address: %p\n", (void *)shmaddr);

// 6. Accept some data from the user and write it to the memory segment
printf("Enter data to write to shared memory: ");
fgets(input, SHM_SIZE, stdin);
// Remove trailing newline if present
input[strcspn(input, "\n")] = '\0';

strcpy(shmaddr, input);

// 7. Display the data entered
printf("Data written to shared memory: %s\n", shmaddr);

// (Optional) Detach but keep segment for reader
// shmdt(shmaddr);

printf("Writer done. You can now run the reader.\n");
printf("Press Enter to remove shared memory...");
getchar();

// Clean up: detach and remove shared memory segment
shmdt(shmaddr);
shmctl(shmid, IPC_RMID, NULL);
printf("Shared memory removed.\n");

return 0;
}
```

Granting Executable permission and Compiling: This section shows that the executable permission has been granted and the program compiled without error.

```
usama@ubuntu:~/Desktop/Lab6/Lab8$ chmod +x writer.c
usama@ubuntu:~/Desktop/Lab6/Lab8$ gcc writer.c -o writer
```

Run and Enter Input: The terminal displays a successful execution in which the writer process stores the message "Hello, this is shared memory!" and the reader process successfully retrieves it.

```
usama@ubuntu:~/Desktop/Lab6/Lab8$ ./writer
Shared memory created with ID: 0
Attached at address: 0x7c408d912000
Enter data to write to shared memory: Hello! This is Lab Task 8.
Data written to shared memory: Hello! This is Lab Task 8.
Writer done. You can now run the reader.
Press Enter to remove shared memory...
Shared memory removed.
```

2. Algorithm for Reading Data from Shared Memory

File creation: Opening the terminal in the directory ~/Desktop/Lab6/Lab8 and using the Vim text editor to create or modify the reader.c source file.

```
usama@ubuntu:~/Desktop/Lab6/Lab8$ vim reader.c
```

Code: Writing the C program, including headers, defining the shared memory key/size, and using **shmget** and **shmat** to access and attach to the memory segment. Also, by adding logic to print the memory address, display the retrieved data string, and safely detach the segment using **shmdt**.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#define SHM_SIZE 1024
#define SHM_KEY 1234 // Must match the key used in writer

int main() {
    // 2. Create necessary variables
    int shmid;
    char *shmaddr;

    // 3. Access the shared memory with the unique key used in the writer
    shmid = shmget(SHM_KEY, SHM_SIZE, 0666);
    if (shmid == -1) {
        perror("shmget");
        exit(EXIT_FAILURE);
    }
    printf("Accessed shared memory with ID: %d\n", shmid);

    // 4. Attach the new process to the shared memory segment
    shmaddr = (char *)shmat(shmid, NULL, 0);
    if (shmaddr == (char *)-1) {
        perror("shmat");
        exit(EXIT_FAILURE);
    }

    // 5. Display the address of the memory segment
    printf("Attached at address: %p\n", (void *)shmaddr);

    // 6. Display the data in that memory segment
    printf("Data read from shared memory: %s\n", shmaddr);

    // Detach (do not remove segment; writer will remove it)
    shmdt(shmaddr);

    return 0;
}
```

Granting Executable permission and Compiling: Modifying file permissions with `chmod +x` to make the file executable and compiling the source code into a binary named `reader` using the GCC compiler.

```
usama@ubuntu:~/Desktop/Lab6/Lab8$ chmod +x reader.c
usama@ubuntu:~/Desktop/Lab6/Lab8$ gcc reader.c -o reader
```

Run and Read data: Executing the compiled `./reader` program, which successfully attaches to the shared memory ID 0 and prints the message: "Hello! This is Lab Task 8."

```
usama@ubuntu:~/Desktop/Lab6/Lab8$ ./reader
Accessed shared memory with ID: 0
Attached at address: 0x78566f3d3000
Data read from shared memory: Hello! This is Lab Task 8.
```